Auto Keyword in C++

Meaning and Usage in Modern C++ (C++11 and later)

The `auto` keyword is used for **type deduction**, meaning the compiler automatically determines the type of a variable from its initializer. This makes code cleaner and easier to write, especially when the type is long or complex.

Example:

```
auto x = 10;      // x is deduced as int
auto y = 3.14;    // y is deduced as double
```

So when you see `auto` in modern C++, it always means **type deduction**.

Old Meaning of auto (Storage Class)

Earlier in C and old C++, `auto` was used as a **storage class specifier**, meaning a variable had **automatic storage duration** (lives only inside the block). But this is now useless and **illegal in modern C++**.

Example (illegal now):

```
auto int x;   // illegal in C++11 onwards
```

Block-scope variables already have automatic storage, so you should simply write:

```
int x;
```

Meaning of "Deduced" in C++

"Deduced" means the compiler **figures out the type automatically** based on the value or expression.

Example with auto:

```
auto x = 42;     // deduced as int
auto y = 3.14;   // deduced as double
```

Example in templates:

```
template<typename T>
void printType(T value)
{
    std::cout << typeid(value).name();
}

printType(42);     // T is int
printType(3.14);   // T is double
```

So "type deduction" simply means **compiler decides the type for you based on context**.

---

const_cast in C++

Meaning

`const_cast` is used to **add or remove const** from a variable's type. It is mainly used when calling functions that do not accept const arguments.

But modifying a variable that is truly const using `const_cast` causes **undefined behavior**.

Safe Case

It is safe only if the original variable was not declared const.

```
void update(int* ptr)
{
    *ptr = 100;
}

int main()
{
    int y = 50;
    const int* p = &y;
    update(const_cast<int*>(p));  // Safe
    cout << y;   // prints 100
}
```

Unsafe Case

If the original data is const, modifying it is not allowed.

```
const int x = 10;
const int* p = &x;
*const_cast<int*>(p) = 20;   // undefined behaviour
```

Program may crash or behave unpredictably.

Summary of const_cast

• Removes or adds const temporarily
• Useful when calling old-style APIs
• Safe only if the original variable was not const
• Dangerous if you modify a value that was declared const

String Example: Mutable vs Read-Only

Mutable String

```
char msg[] = "Hello";
```

This creates a **modifiable character array on stack**.

```
msg[0] = 'J';   // valid
cout << msg;    // Jello
```

Read-Only String Literal

```
const char* msg = "Hello";
```

This points to a **string literal stored in read-only memory**.
You can read it, but modifying it is not allowed.

---

constexpr in C++

Meaning

A **constexpr expression** is:

• A value that does not change
• Can be evaluated at compile time

`constexpr` was introduced in C++11 to allow compile-time computation and avoid runtime overhead.

constexpr Function Example

```
constexpr int fun2(int k)
{
    return k * k;
}
constexpr int result2 = fun2(3);
int arr[result2];   // valid
```

A normal function result cannot be used like this.

```
int fun1(int k)
{
    return k * k;
}

int result1 = fun1(3);
// int arr[result1];   // not allowed
```

constexpr Constructor Example

```
class Number
{
    int a, b;
public:
    constexpr Number(int a, int b) : a(a), b(b) {}
};
constexpr Number n2(400, 500);
```

Such objects can be evaluated at compile time.

---

decltype in C++

`decltype` deduces the **type of an expression at compile time**.

Basic Example

```
int x = 5;
decltype(x) y = 10;      // y is int
```

Function Return Type

```
int foo();
decltype(foo()) var;   // same type as foo return
```

Expression Type Deduction

```
int a = 5;
double b = 10.5;
decltype(a + b) sum;   // sum is double
```

Array Element

```
int arr[] = {1,2,3};
decltype(arr[0]) elem = arr[0];   // elem is int
```

Reference Deduction

```
int x = 5;
int& ref = x;
decltype(ref) another_ref = x;   // another_ref is int&
```

---

auto vs decltype

When auto is sufficient

Use `auto` when:

• There is an initializer
• Type can be inferred directly
• You want clean and simple code

Examples:

```
int a = 42;
auto b = a;      // b = int
auto add(int x, int y)
{
    return x + y;   // return type deduced
}
std::vector<int> v = {1,2,3};
auto it = v.begin();   // iterator type deduced
```

When decltype is needed

Use `decltype` when:

• You need the exact type of an expression
• Type may be reference
• const needs to be preserved
• There is no initializer

Example where auto fails but decltype works:

```
int x = 10;
int& r = x;

auto a = r;         // a becomes int (reference lost)
decltype(r) b = x; // b remains int&
```

`decltype` preserves exact type.

Delegating Constructor in C++

In C++11, one constructor in a class can call another constructor of the same class. This is called a **delegating constructor**.

One constructor acts as the **target constructor**, and the other delegates work to it.

Example:

```
class Number
{
    int num;
public:
    Number(int num)
    {
        this->num = num;
    }

    Number() : Number(1)   // delegating constructor
    {
    }

    int getNum()
    {
        return num;
    }
};
```

When `Number()` runs, it internally calls `Number(1)`.

So:

```
Number n1{12};    // num = 12
Number n2;        // num = 1
```

Purpose:

• Avoids code duplication
• Keeps initialization logic in one place

---

enum class (Scoped Enumeration)

`enum class` was introduced in C++11.

It is strongly typed and scoped, meaning its values do not convert automatically to int and cannot mix with other enums.

Example:

```
enum class Color
{
    RED,
    GREEN,
    BLUE
};
```

Usage:

```
Color c = Color::RED;
```

You must compare using scope:

```
if (c == Color::GREEN)
```

It cannot convert to int automatically, so you must cast:

```
int x = static_cast<int>(c);
```

Benefits:

• Prevents accidental misuse
• No name conflicts
• Provides strong type safety

---

final Class in C++

A `final` class **cannot be inherited**.

Example:

```
class A final
{
public:
    virtual void show(int a)
    {
        cout << a;
    }
};
```

```
class B : public A   // error
{
};
```

Use final when you do not want others to derive subclasses from it.

---

final Method in C++

A virtual method marked as `final` cannot be overridden in derived classes.

Example:

```
class A
{
public:
    virtual void show(int a) final
    {
        cout << a;
    }
};
```

If a derived class tries:

```
class B : public A
{
public:
    void show(int k)   // error
    {
    }
};
```

This gives a compile-time error because overriding is not allowed.

Note:
Method hiding with different signature is still allowed.

Deleted Function vs Private Function

Deleted Function (= delete)

A deleted function **cannot be called at all**.
Not inside the class.
Not outside the class.

It is commonly used to disable:

• Copy constructor
• Assignment operator
• Other unwanted operations

It clearly tells the compiler: **this function must never be used.**

Example:

```
class Test
{
public:
    Test() = default;
    Test(const Test&) = delete;
};
```

Any call to copy the object gives a compile time error.

Even the class itself cannot use that function.

Also, once a function is deleted, changing access specifiers cannot make it valid again.

You cannot write any code inside a deleted function, because `= delete` is not a function body. It is simply a compiler instruction.

So the answer is:

No, you cannot write code inside a function marked = delete.

Private Function

A private function:

• Can be called from inside the class
• Cannot be called from outside the class

It is used for **encapsulation**, not restriction.

It still exists and works normally inside the class.

So:

Deleted = function is completely banned
Private = function works, but only inside the class

Lambda Expressions in C++

Lambda functions are **inline anonymous functions** introduced in C++11.
They are useful for short, temporary functionality without creating a named function.

General structure:

```
[] (parameters) {
    body
};
```

Meaning of parts:

- `[]` is the lambda introducer
- `()` is the parameter list
- `{}` is the function body

Example:

```
auto fun = [](){
    cout << "Hello";
};
```

Calling it:

```
fun();
```

Variable Capture in Lambda

By default, lambda cannot access outer function variables unless they are captured.

There are two main capture styles.

Capture by Value

Copies the variable.

```
int a = 10;
auto f = [a]() {
    cout << a;
};
```

Capture by Reference

Uses the original variable.

```
int a = 10;
auto f = [&a]() {
    a++;
};
```

Common Capture Forms

- `[ ]` – capture nothing
- `[=]` – capture all variables by value
- `[&]` – capture all variables by reference
- `[a]` – capture only a by value
- `[&a]` – capture only a by reference

Why Lambdas Are Useful

• Short and clean
• Avoid writing extra functions
• Very useful in STL algorithms like sort, find_if, etc.

Example:

```
std::sort(vec.begin(), vec.end(),
          [](int a, int b){
              return a < b;
          });
```

Move Semantics in C++

Move semantics allow transferring ownership of resources from one object to another, instead of copying them. This is most useful when objects manage resources such as heap memory, file handles, buffers, containers, etc.

For **primitive types like int**, move behaves like copy because there is no owned resource to transfer.

Example explanation:

```
MyClass m3 = move(m1);
```

The move constructor is called. But since `num` is just an int, the value is simply copied. The source object `m1` still holds the same value. Moving really matters for objects that manage resources, not simple values.

Compiler-Generated Functions in Modern C++

When you do not define constructors, the compiler automatically provides:

• Default constructor
• Copy constructor
• Copy assignment operator
• Move constructor (C++11 onward)
• Move assignment operator (C++11 onward)
• Destructor

The compiler only generates these if applicable.

Suppression Rules

If you explicitly define a **copy constructor**, the compiler does **not** create a move constructor.

If you explicitly define a **move constructor**, the compiler does **not** create a copy constructor.

If you define a **move assignment operator**, the compiler does **not** generate a normal assignment operator.

So once you start customizing special member functions, you may have to define the others yourself if needed.

---

nullptr in C++

`nullptr` was introduced in C++11 as a **type-safe null pointer constant**.
It is of type `std::nullptr_t`.

Unlike `0` or `NULL`, it is never treated as an integer.

Example:

```
void disp(int* ptr)
{
    cout << "inside pointer function";
}
void disp(int k)
{
    cout << "inside int function";
}
```

Calling:

```
disp(nullptr);
```

This always calls the pointer version.
If you pass `NULL`, behavior may depend on compiler, making it unsafe and non-portable.

So `nullptr` removes ambiguity.

Rvalue and Rvalue References

An **rvalue** is a temporary value or literal such as `10`, `x+3`, a function return by value, etc.

An **lvalue** is a named object with a memory location such as `x`.

Rvalue Reference Example

```
int&& r = 10;
```

`r` references the temporary value `10`.
You can modify it:

```
r = 20;
```

Rvalue references exist to support move semantics.

Function Overloading with References

```
void fun(int& r);            // lvalue reference
void fun(const int& r);      // const lvalue reference
void fun(int&& r);           // rvalue reference
```

The matching priority is:

• exact lvalue reference
• then rvalue reference
• then const reference

`std::move(x)` converts an lvalue into an rvalue reference, meaning
"treat this object as a temporary".

Example:

```
fun(move(i));
```

Binding Rules

An rvalue reference cannot bind to a normal variable:

```
int x = 10;
int&& ref = x;   // error
```

But this works:

```
int&& ref = std::move(x);
```

Because `move(x)` tells compiler to treat `x` as a temporary.

Special Case: const lvalue reference

A const lvalue reference can bind to an rvalue:

```
const int& a = 10;
```

So:

• `int&` → binds only lvalues
• `const int&` → binds both
• `int&&` → binds only rvalues

Practical Move Example with std::string

```
string s1 = "Hello";
string s2 = std::move(s1);
```

After move:

• s2 owns the data
• s1 becomes empty or unspecified

Copy Constructor vs Move Constructor

Copy constructor:
- Makes a new object by copying data
- Usually deep copy
- Can be expensive

Move constructor:
- Transfers ownership of resources
- Avoids copying
- Very fast for big objects

Copy is for duplication.
Move is for efficient transfer.

---

lvalue vs rvalue Summary

lvalue:

- Has memory location
- Exists beyond expression
- Can appear on left side of assignment

Examples:

```
int x;
x = 5;
int& ref = x;
```

rvalue:

- Temporary
- No persistent storage
- Appears on right side

Examples:

```
5
x + 3
funReturningInt()
```

Move semantics mainly apply to **rvalues**.

Key Summary of Reference Types

- `int&` → lvalue reference
- `const int&` → can bind to both
- `int&&` → rvalue reference

Rvalue reference is designed for temporaries and move semantics.

Smart Pointers in C++

Smart pointers are special classes in C++ that manage dynamically allocated memory automatically. They help avoid memory leaks, dangling pointers, and manual delete calls.

Advantages of Smart Pointers

They manage resources safely.
They help avoid memory leaks.
They prevent dangling pointers.
They provide clear ownership of objects so only allowed owners access the resource.
They simplify memory management because you no longer need to remember when to call delete.
They help avoid manual memory-management mistakes.

---

unique_ptr

A `unique_ptr` represents **exclusive ownership** of a resource.
Only one `unique_ptr` can point to a resource at a time.

When it goes out of scope, it deletes the object automatically.

Key ideas:

• Cannot be copied
• Can be moved
• Lightweight
• Used for exclusive ownership

Example:

```
unique_ptr<MyClass> ptr1 = make_unique<MyClass>();
ptr1->show();

// unique_ptr<MyClass> ptr2 = ptr1;   // Error
unique_ptr<MyClass> ptr2 = move(ptr1);   // Ownership transferred
```

After move:

`ptr1` becomes null.
`ptr2` owns the object.

---

shared_ptr

A `shared_ptr` allows multiple smart pointers to share ownership of the same resource.

The resource is destroyed when the **last** shared_ptr releases it.

Key ideas:

• Maintains reference count
• Deletes object when count becomes zero
• Slightly slower due to counting

Example:

```
shared_ptr<MyClass> ptr1 = make_shared<MyClass>();
shared_ptr<MyClass> ptr2 = ptr1;

cout << ptr1.use_count();
```

Both pointers share the same object.

When to use which?

Use unique_ptr when:

You want strict ownership.
Only one owner should exist at a time.
You want maximum performance with minimum overhead.

This means:
You get safety without the cost of reference counting.

Use shared_ptr when:

Multiple parts of code need to share the same object.
Object should be deleted only when the last owner is done.

---

static inline (C++17)

Before C++17, static class members had to be defined separately outside the class.
If not, you would get a linker error.

Example:

```
class A
{
public:
    static int cnt1;
};

int A::cnt1 = 10;
```

From **C++17 onward**, you can define and initialize static members inline:

```
class B
{
public:
    static inline int cnt2 = 10;
};
```

No separate definition is required.

This works only when compiling with C++17 or later.

---

std::array Container

`std::array` was introduced in C++11.

It is like a normal fixed-size array but behaves like a container.
Size must be known at compile time.

Benefits:

• Fixed size
• Faster than vector
• Supports STL functions
• Stores data in stack like normal array

Example:

```
array<int,3> a1 = {11,2,13};
sort(a1.begin(), a1.end());
```

You can:

• sort
• access elements
• get size
• iterate using range-based loop

Example outputs include size, first and last elements, etc.

You cannot change size after creation.
You can assign new values to elements.
